CYBERSECURITY · HOST INTRUSION DETECTION SYSTEM

HIDS Sentinel

An Intelligent Host Security Monitoring System

Host Intrusion Detection System — Sentinel Edition

A real-time security monitoring dashboard that watches over your Windows computer 24 hours a day — detecting threats, analysing suspicious behaviour with Artificial Intelligence, scanning firmware and hardware, and giving you a simple security grade so you always know how safe your machine is — all from a web browser.

19	8	3	0–100
CORE MODULES	SECURITY SCANNERS	AI / ML LAYERS	POSTURE SCORE

Course: Tools & Techniques for Data Science | **Academic Year:** 2025–26 | **Team:** 3 Members

Document Type: Detailed Project Report (General Audience)

Confidential — For Academic Use

Team Member Roles & Responsibilities

This project was designed, built, and tested by a team of three members. The project is split into three functional areas — Backend & Security Core, Machine Learning & Dashboard, and Security Scanning & Research. Each member owns their area end-to-end and will present it during the project demonstration.

MEMBER 1

Backend Engineer & System Architect

Core engine · API routes · Authentication · Database

→ Overall system design and architecture decisions

→ FastAPI REST API — all 40+ endpoints

→ User login, registration, JWT token security

→ Password hashing with bcrypt

→ MongoDB Atlas database integration

→ Real-time system monitor (CPU/RAM/disk/network)

→ WebSocket broadcaster (live data push)

→ SOAR automated response module

→ UAC-elevated firmware scanner

→ Rate limiting and API security hardening

📋 **Presents: Sections 1, 2, 9 — Architecture, Modules & How It Works**

MEMBER 2

ML Engineer & Frontend Developer

Machine Learning · Telemetry · Dashboard UI

→ React.js frontend — entire dashboard design

→ All UI components, charts, and live graphs

→ Isolation Forest ML anomaly detection

→ Google Gemini AI integration for process scoring

→ Statistical baseline threat detection (Z-score)

→ Real-time telemetry data streaming

→ Alert management and event analytics

→ Deep scan results UI with clickable findings

→ MITRE ATT&CK mapping in alerts

→ PDF/DOCX report generation module

📋 **Presents: Sections 6, 8 — Telemetry & Machine Learning**

MEMBER 3

Security Scanner & Research Analyst

Threat scanning · YARA · Windows internals · Research

→ Deep hardware/firmware security scanner

→ YARA malware signature scanning engine

→ Startup entry & persistence mechanism detection

→ Windows Registry and BIOS/UEFI inspection

→ Network port and connection analyser

→ File Integrity Monitoring (FIM) module

→ Windows Event Log reader & analyser

→ Market research and competitor analysis

→ Real-world use-case documentation

→ Future roadmap planning

📋 **Presents: Sections 3, 4, 5, 7 — Scanners, Market & Use-Cases**

How to Read This Report

📖 **No Technical Background Required**

This document is written for a reader who has no knowledge of cybersecurity or software development. Every technical term is explained using everyday analogies before being defined formally. Coloured boxes highlight key points, tables summarise comparisons, and diagrams show how the system flows. You do not need to understand code to understand this report.

1. Project Summary

> *"Every 39 seconds, a cyberattack occurs somewhere in the world. Most victims never know their computer was compromised until the damage is already done — files encrypted, passwords stolen, data sold. HIDS Sentinel is designed to catch these threats in real-time, before the damage happens."*

The Problem This Project Solves

Today, almost everyone uses a computer for banking, personal communication, business operations, or storing important documents. Yet most people have only a basic antivirus program protecting them — software that only recognises threats it has already seen before. New malware, firmware-level attacks, and sophisticated hacking tools are designed specifically to evade traditional antivirus.

Professional-grade security monitoring tools (called SIEM systems — Security Information and Event Management) exist, but they cost hundreds of thousands of rupees per year, require a dedicated IT department to operate, and are designed for large corporations — not individuals, students, clinics, or small businesses.

The Solution — HIDS Sentinel

HIDS stands for **Host Intrusion Detection System**. "Host" refers to a single computer (your machine). "Intrusion Detection" means automatically discovering signs that something unauthorised is happening on that machine. "Sentinel" means a guard who stands watch continuously.

HIDS Sentinel brings enterprise-level security monitoring to anyone who can open a web browser. It is installed once on a Windows computer, runs as a background service, and you access the security dashboard from any browser — on the same machine or from another device on your network.

✅ What HIDS Sentinel gives you in plain English

- A **live security score (0–100)** telling you how safe your computer is right now
- A **real-time dashboard** showing CPU, memory, disk, and network usage as live graphs
- Automatic detection of **malware, suspicious programs, and hacker tools**
- A **deep hardware scan** that checks your BIOS chip, TPM security module, and Secure Boot
- A full list of **every program that starts automatically** when your computer boots
- **AI-powered analysis** of running processes — flagging anything that behaves abnormally
- Downloadable **PDF security reports** you can share with your IT support person

The Security Posture Score — Your Machine's Grade

The most visible feature of HIDS Sentinel is the **Security Posture Score** — a single number between 0 and 100 that tells you how secure your machine is, like a school grade:

90–100	75–89	55–74	35–54	0–34
Grade A	**Grade B**	**Grade C**	**Grade D**	**Grade F**
Excellent. Minimal risks found.	Good. Minor issues to address.	Fair. Several risks detected.	Poor. Multiple serious issues.	Critical. Immediate action required.

Each security finding deducts points from a starting score of 100. For example, a missing TPM chip might deduct 5 points, a suspicious startup program deducts 10 points, and Secure Boot being disabled deducts 15 points. Every finding comes with a plain-English "Fix" instruction so you know exactly what to do.

Who Built This and Why?

This project was built as part of an academic course on Data Science tools. The team chose a cybersecurity project because it naturally combines multiple disciplines — data streaming (telemetry), machine learning (anomaly detection), web development (the dashboard), database management (MongoDB), and systems programming (reading Windows internals). It is a real, working tool — not a simulation — that produces actual security data from the computer it runs on.

3. What This Project Actually Does — A Complete Walkthrough

HIDS Sentinel works like a team of specialised security guards, each watching a different area of your computer simultaneously. Here is exactly what happens from the moment you start the system:

1 **Server Starts — MongoDB Connects & Baseline Loads**

The Python backend (FastAPI) starts up, connects to MongoDB Atlas database, loads the last 1,000 security events to build a behavioural baseline, and initialises all 8 security scanners. The Isolation Forest ML model is trained on synthetic baseline data in memory.

2 **You Open the Browser Dashboard & Log In**

You go to `http://localhost:8000` in your browser. A React.js application loads — this is your security cockpit. You log in with your username and password. The server verifies your identity using bcrypt password hashing and issues a JWT token (a temporary digital pass valid for 60 minutes) so you don't have to log in on every action.

3 **Live Telemetry Begins — Dashboard Updates Every 3 Seconds**

The browser opens a WebSocket connection — a permanent live channel — to the server. Every 3 seconds, the monitor module reads your CPU usage, RAM usage, disk read/write speed, and network upload/download speed and pushes this data to your browser. The line graphs on the dashboard update in real-time without you refreshing the page.

4 **You Click "Start Deep Scan" — A Full System Inspection Begins**

The Deep System Scanner runs — this takes 20–45 seconds. It inspects: BIOS firmware version, TPM chip status, Secure Boot setting, all startup programs, scheduled tasks, open network ports, every running process (checked against ML and known-bad lists), USB devices, and Windows security settings.

5 **Results Appear — Posture Score Calculated**

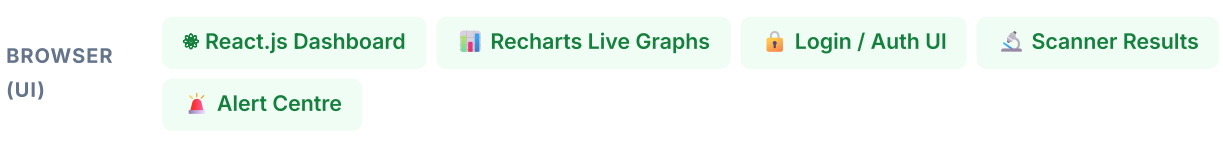
Each finding is scored and deductions are applied. The final score (0–100) and grade appear. Every finding card is **clickable** — clicking it opens a popup listing the exact suspicious items (e.g. the specific startup entry by name, path, and registry location). You can then switch between 9 tabs: Posture Score, Machine Info, Firmware/BIOS, Hardware, Security, Persistence, Network, Processes, and Devices.

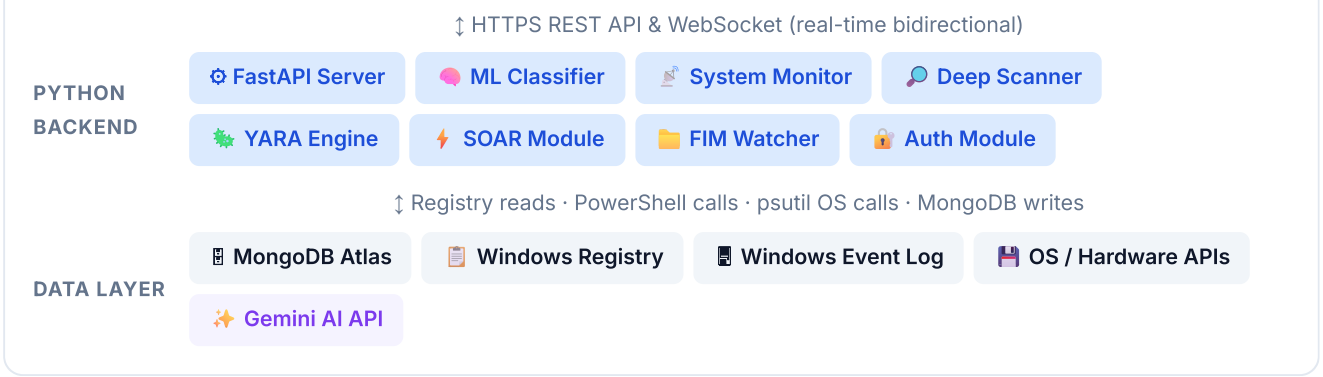
6 **Events Are Logged — Alerts Fire Automatically**

If any activity crosses the threat threshold (score ≥ 35), an alert is created and stored in MongoDB. Critical alerts (score ≥ 75) also trigger the SOAR module which can automatically take action — like terminating a malicious process. All events and alerts remain searchable in the History page.

9. Project Architecture — How Everything Connects

The system has three distinct layers that communicate with each other:





🔄 Data Flow in Plain English

Your browser asks the Python server for information → The Python server reads your computer's real data (process list, ports, registry keys, hardware) using Windows system calls → It analyses the data using ML and security rules → It saves important events to MongoDB → It sends the results back to your browser → Your dashboard updates. This cycle happens continuously for live metrics and on-demand for scans.

2. Technical Modules — What Each Part of the System Does

A module (also called a file or component) is like a department inside a company — each one has a specific job and talks to the others when needed. HIDS Sentinel has 19 Python modules and a complete React.js frontend. Here is what each one does, explained without jargon:

Module	What It Does (Plain English)	Key Technology
main.py	**The front door.** Starts the entire server. Connects to MongoDB, runs startup checks, initialises all other modules, and begins the background monitor loop. Everything starts from here.	`FastAPI, asyncio, uvicorn`
monitor.py	**The health monitor.** Continuously reads CPU usage, memory (RAM), disk speed, network speed, system uptime, and active process count every 3 seconds. Caches the latest reading so any connected browser can get it instantly without waiting. Also stores hardware metadata (CPU model, total RAM, manufacturer) at startup by reading the Windows Registry — no internet needed.	`psutil, winreg, asyncio`
system_scanner.py	**The deep inspector.** The largest module (900+ lines). Performs a comprehensive security scan: reads BIOS manufacturer and version from the Windows Registry, checks the TPM chip using PowerShell's Get-PnpDevice command (works without admin rights), verifies Secure Boot state, lists all startup programs from the Registry, enumerates all scheduled tasks, lists every open network port and which program is using it, analyses every running process, lists USB/PnP devices, and computes the final security posture score with grade.	`winreg, psutil, subprocess, PowerShell`
elevated_scanner.py	**The privileged hardware scanner.** Some Windows security data (like TPM chip ownership and full Secure Boot state) is locked behind Administrator access. This module requests that access by triggering a Windows UAC dialog (the blue "Do you want to allow this?" popup). Once approved, it runs an elevated PowerShell script that reads Get-Tpm, Confirm-SecureBootUEFI, Win32_Tpm, and Windows Defender status, writes the result to a temporary file in %TEMP%, and the Python backend reads and deletes the temp file. Nothing is permanently stored in elevated mode.	`ctypes, ShellExecuteW, PowerShell, tempfile`
ai_classifier.py	**The AI threat analyst.** Contains the two-layer AI system. Layer 1 uses scikit-learn's Isolation Forest ML model to detect abnormal process behaviour by analysing 6 features (CPU, memory, command length, risk flags). Layer 2 sends suspicious processes to Google's Gemini 2.5 Flash AI for a second opinion with contextual analysis. If no API key is configured, a rule-based fallback engine still provides threat categorisation.	`scikit-learn, numpy, Gemini API (gemini-2.5-flash)`
detector.py	**The scoring engine.** When a security event is submitted (e.g. a failed login), this module calculates a danger score using two methods combined: (1) rule-based weights (a brute-force event starts at 18 points, privilege escalation at 35 points), and (2) statistical anomaly scoring comparing the current event's metrics against the baseline average from the last 1,000 events. The two scores are combined (45% anomaly + 75% rule weight) to produce a final severity: Low / Medium / High / Critical.	`Python statistics (mean, stdev), MITRE ATT&CK mapping`

Module	What It Does (Plain English)	Key Technology
yara_scanner.py	**The malware fingerprinter.** Compares files or text strings against YARA rules — pattern definitions that describe the unique code signatures of known malware. Built-in rules detect Mimikatz (credential stealer), ransomware file encryption patterns, Cobalt Strike beacons (professional hacker toolkit), and web shells (backdoors placed in web servers). Returns match name, severity, description, and MITRE tags.	`YARA rules, pattern matching`

Module	What It Does (Plain English)	Key Technology
auth.py & mongo_auth.py	**The security door.** Handles everything related to user accounts. When you register, your password is hashed using bcrypt (a one-way transformation — the original password can never be recovered from the hash, even if the database is stolen). When you log in, the submitted password is hashed and compared to the stored hash. If they match, a JWT (JSON Web Token) is issued — a digitally signed, time-limited pass that proves your identity on every subsequent API request without re-entering your password. Rate limiting (5 failed attempts = temporary lockout) prevents brute-force attacks.	`bcrypt`, `python-jose, JWT`, `slowapi`, `MongoDB`
fim.py	**The file guardian.** File Integrity Monitoring creates SHA-256 "fingerprint" hashes of important system files. SHA-256 means if even a single character in a file changes, the fingerprint completely changes — making tampering immediately detectable. If a watched file is modified by anything other than Windows Update, an alert fires. This is how ransomware is caught in its earliest stage — the moment it starts changing files.	`watchdog`, `hashlib (SHA-256)`, `asyncio`
os_event_log.py	**The Windows diary reader.** Windows secretly records everything in its Security Event Log — failed logins, account lockouts, new services being installed, system shutdowns. This module reads those logs using pywin32 (Python's Windows API bridge) and filters for security-relevant events. Each event is mapped to a MITRE ATT&CK technique code — the internationally standardised way that professional security teams categorise hacker tactics (e.g. T1110 = Brute Force, T1068 = Privilege Escalation).	`pywin32`, `Windows Security Event Log`, `MITRE ATT&CK`
soar.py	**The auto-responder.** SOAR stands for Security Orchestration, Automation and Response. When a Critical alert fires, SOAR can take automated action without waiting for a human. Possible actions include: terminating a malicious process, adding a firewall block rule for a suspicious IP address, or locking a user account after too many failed logins. Think of it as the system calling the fire department automatically when smoke is detected, rather than waiting for a person to notice and make the call.	`subprocess`, `Windows Firewall API, psutil process management`
broadcaster.py	**The live news ticker.** Manages WebSocket connections — the permanent live channels between the server and each connected browser. When a new alert is created or metrics update, the broadcaster pushes this information to all connected browsers simultaneously. This is what makes the dashboard "live" — graphs update without you pressing refresh, and alert notifications appear the moment a threat is detected.	`FastAPI WebSocket`, `asyncio`
agent_manager.py	**The network coordinator.** Manages lightweight "agent" programs that can be installed on other computers in the same network. Each remote agent checks in with the central HIDS Sentinel server, reporting its IP address, hostname, operating system, and online status. This is the foundation for monitoring multiple machines from one dashboard (currently in early development).	`MongoDB`, `asyncio`, `HTTP heartbeat`
storage.py	**The long-term memory.** Handles all reads and writes to the MongoDB database. Saves every security event, alert, host snapshot, and scan result permanently so you can look back at the history of what happened on your computer days or weeks later. Uses Motor — an asynchronous MongoDB driver — so database operations don't slow down the live dashboard.	`Motor (async MongoDB)`, `pymongo`

Module	What It Does (Plain English)	Key Technology
reporting.py	**The report printer.** Takes the current set of alerts, events, and scan results and packages them into a professionally formatted downloadable PDF or Word (DOCX) document. Security managers can share these reports with executives or compliance auditors. The PDF includes a summary page, alert table, event timeline, and recommendations.	`ReportLab (PDF), python-docx (Word)`
security.py	**The security utilities..** A helper library with security-focused functions used by other modules — input validation (preventing malicious data being submitted through the API), sanitisation functions, and security configuration constants (e.g. JWT expiry times, allowed CORS origins, algorithm settings).	`python-jose, FastAPI security utilities`
simulator.py	**The test event generator.** Used during development and demonstrations to generate realistic-looking security events (failed logins, process starts, file changes) so the system can be tested without waiting for real threats to occur. Generates events that match the same format as real Windows events.	`Python random, faker-style generation`

5. The Scanners — Full Names, How They Work & What They Return

Each scanner is an independent inspection engine. Here is a complete description of all 8 scanners:

🔬 Scanner 1: Deep System Security Scanner

Full name: Deep Hardware and Operating System Security Scanner

This is the flagship scanner — the most comprehensive inspection the system performs. It is the equivalent of taking your car for a full MOT service, but for your computer's security. When you click "Start Deep Scan", the system spends 20–45 seconds reading data from 9 different areas of your computer simultaneously:

- **Firmware/BIOS:** Reads the BIOS manufacturer (e.g. American Megatrends / AMI), model, version string, and release date from the Windows Registry. Older BIOS versions may have unpatched vulnerabilities.
- **TPM Chip:** Checks if a Trusted Platform Module hardware chip is present and what version it is (1.2 or 2.0). TPM 2.0 is required for Windows 11 and provides hardware-level encryption.
- **Secure Boot:** Verifies whether Secure Boot is enabled — a UEFI feature that prevents unauthorised operating systems from loading during startup (blocks bootkit malware).
- **Startup Programs:** Reads every program registered to auto-start from the Windows Registry (HKLM\Run, HKCU\Run) and the Startup folder, flags unknown or unsigned entries.
- **Scheduled Tasks:** Lists all tasks scheduled to run automatically, highlighting any created by non-Microsoft publishers.
- **Open Network Ports:** Lists every TCP/UDP port your computer is listening on with the associated process name.
- **Running Processes:** Lists every currently running program with PID, CPU/memory usage, and AI-powered suspicion assessment.
- **USB/PnP Devices:** Lists all connected hardware devices.
- **Windows Security:** Checks Windows Defender and firewall status.

Output: A Security Posture Score (0–100), a Grade (A–F), and a list of findings — each with a severity label (Critical/High/Medium/Low), deduction amount, plain-English description, and a specific fix recommendation. Every finding card is clickable to show the exact suspicious items.

🦠 Scanner 2: YARA Malware Signature Scanner

Full name: Yet Another Recursive Acronym — Malware Signature Pattern Scanner

YARA is an industry-standard tool used by antivirus companies and government cybersecurity agencies worldwide. It works like a fingerprint database. Every piece of malware has unique patterns in its code — specific sequences of bytes, text strings, or structural characteristics that identify it. These patterns are described in "YARA rules" — text files that define what to look for.

The HIDS Sentinel YARA scanner includes built-in rules for 4 malware families:

- **Mimikatz** — A password extraction tool used by hackers to steal Windows credentials from memory. It is detected by its unique internal strings and function names.
- **Ransomware** — Detected by patterns common to ransomware: ransom note text strings ("your files have been encrypted"), encryption function signatures, and file extension manipulation patterns.
- **Cobalt Strike** — A professional penetration testing toolkit often misused by attackers. Detected by its shellcode beacon patterns and unique API call sequences.
- **Web Shells** — Backdoor scripts attackers plant on web servers. Detected by PHP/ASP/JSP code patterns that execute system commands from web requests.

Output: For each file or string scanned: a Clean/Malicious verdict, matched rule name, severity rating, text description of the threat, and MITRE ATT&CK technique code.

🤖 Scanner 3: AI Process Intelligence Scanner

Full name: AI-Enhanced Running Process Security Inspector

This scanner examines every program currently running on your computer. For each process, it extracts: name, full command-line arguments, CPU usage %, memory usage %, and whether the executable is digitally signed. It then runs the process data through three analysis layers (explained fully in Section 8) and returns a confidence score (0–100%) for each suspicious process along with a human-readable reason.

🔐 Scanner 4: Firmware & TPM Elevated Scanner

Full name: UAC-Elevated Firmware and Trusted Platform Module Inspector

Windows locks certain hardware data behind Administrator-level access. This scanner requests that access by triggering a Windows UAC (User Account Control) dialog — the standard "Do you want to allow this app to make changes to your device?" blue popup. Once the user clicks "Yes", an elevated PowerShell script runs that accesses Get-Tpm (TPM state), Confirm-SecureBootUEFI (actual enforcement status), Win32_Tpm CIM class (manufacturer), and Get-MpComputerStatus (full Windows Defender configuration including signature age, last scan time, and all protection settings). Results are written to a temporary JSON file, read by the Python backend, and the temp file is immediately deleted. Nothing elevated remains.

🚀 Scanner 5: Persistence Mechanism Scanner

Full name: System Startup & Persistence Mechanism Inspector

"Persistence" in cybersecurity means a hacker's ability to keep their malware running even after the computer is rebooted. The most common way to achieve this is to register a malicious program to start automatically every time Windows boots. This scanner examines every known persistence location:

- **HKLM\Run and HKCU\Run:** Windows Registry keys where programs register to auto-start. HKLM affects all users on the machine; HKCU affects only the current user. These are the most common places malware hides.
- **Startup Folders:** Windows file system folders (one per user, one for all users) where shortcuts to programs can be placed to make them auto-start.
- **Scheduled Tasks:** The Windows Task Scheduler can run programs at specific times or events. Attackers use this to run malware at login, at a specific time, or on a trigger.

For each entry found, the scanner records: the program name, full path/command, the registry key or folder location, and whether it looks suspicious (e.g. running from a temp directory, using unusual arguments, or having an unrecognised publisher name).

Output: A clickable finding card showing the count of suspicious entries. Clicking it opens a popup listing every suspicious startup item by name, path, and registry location — highlighted in red.

🌐 Scanner 6: Network Port Analyser

Full name: Active Network Connection and Open Port Analyser

Every open port on your computer is like an unlocked door. Port numbers range from 0 to 65,535 — some are standard and expected (port 443 for HTTPS websites, port 80 for HTTP), but others are suspicious if open unexpectedly. This scanner lists every TCP and UDP port that your computer is listening on, along with the process name responsible and whether it is bound to all network interfaces (public-facing) or just the local machine. Unusual high-numbered ports, or known malware command-and-control ports, are flagged immediately.

📁 Scanner 7: File Integrity Monitor (FIM)

Full name: Real-Time File System Integrity Monitoring System

This scanner works differently from the others — it runs continuously in the background rather than on demand. Using Python's watchdog library, it watches a list of important directories (system files, configuration files) for any changes. When a file is created, modified, or deleted, the event is immediately logged and compared against a hash database. The SHA-256 hash of each file is like its unique fingerprint — if the fingerprint changes, the file has been altered. This is the earliest possible warning of ransomware (which modifies files to encrypt them) or rootkit installation (which modifies system files to hide itself).

📋 Scanner 8: Windows Event Log Analyser

Full name: Windows Security Event Log Reader and Threat Classifier

Windows maintains a detailed journal of everything that happens on your computer in its Event Log. Most of these entries are routine, but hidden among them are security-critical events:

- **Event ID 4625** — A login attempt failed. Multiple failures in a short time = brute-force attack.
- **Event ID 4720** — A new user account was created. Was it authorised?
- **Event ID 4672** — Special privilege (Administrator) was assigned. Was this expected?

- **Event ID 7045** — A new Windows service was installed. Attackers often install services to gain persistent admin access.
- **Event ID 4648** — Login using explicit credentials (password was typed). This can indicate an attacker using stolen credentials.

The scanner pulls these events, converts the Windows numeric codes to readable descriptions, and maps each to a MITRE ATT&CK technique code — the global standard used by security professionals.

> **What is MITRE ATT&CK?**
>
> MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) is a publicly available knowledge base maintained by the MITRE Corporation and used by cybersecurity teams worldwide. It catalogues every known technique that real hackers use to compromise computers, broken down into 14 tactic categories (like Initial Access, Privilege Escalation, Exfiltration). Each technique has a unique code (T1110 = Brute Force, T1059 = Command Execution, etc.). HIDS Sentinel automatically tags every alert with the relevant ATT&CK codes so that any security analyst reviewing the alerts immediately understands what type of attack was attempted.

6. What is Telemetry and How Does It Work in HIDS Sentinel?

> **What is Telemetry — The Simple Explanation**
>
> Telemetry means automatically collecting measurements from a remote system and sending them somewhere else to be monitored, without anyone manually taking those readings. The word comes from Greek: "tele" (from afar) + "metron" (measurement). A Formula 1 racing car sends speed, engine temperature, tyre wear, and brake temperature data to the team's garage in real-time — that is telemetry. An aircraft's black box is a telemetry recorder. A weather station sending hourly temperature readings to a server is telemetry. In HIDS Sentinel, **your computer is the data source** and **your browser dashboard is the monitoring station**.

How HIDS Sentinel Collects Telemetry

Every **3 seconds**, the `monitor.py` module runs a background task (called an async coroutine) that:

1. Calls `psutil.cpu_percent()` — reads CPU usage from the operating system kernel
2. Calls `psutil.virtual_memory()` — reads RAM usage including total, used, and available
3. Calls `psutil.disk_io_counters()` — reads cumulative disk read and write byte counts, then calculates speed per second by comparing to the previous reading
4. Calls `psutil.net_io_counters()` — same approach for network upload/download speed
5. Calls `psutil.boot_time()` — calculates how long the system has been running (uptime)
6. Packages all readings into a JSON snapshot and pushes it to the WebSocket broadcaster
7. The broadcaster immediately sends the snapshot to every connected browser
8. The browser receives the data and updates the live graphs without any page refresh

What Telemetry Data is Collected and Why It Matters for Security

Metric	What It Measures	Security Significance (What to Watch For)
CPU Usage (%)	Percentage of processor time being used right now	Sudden sustained 100% CPU: often indicates crypto-mining malware or ransomware encryption process running
Memory / RAM (%)	How much of your computer's working memory is occupied	Rapid memory increase: may indicate a memory injection attack or a program loading a large malicious payload into RAM
Disk Write Speed (MB/s)	How fast data is being written to your storage drive	Sudden high disk write with no expected activity: ransomware encrypting files causes extremely high disk write rates
Disk Read Speed (MB/s)	How fast data is being read from your drive	High disk read with high network upload: classic indicator of data exfiltration (stealing files off your machine)
Network Upload (MB/s)	How much data is leaving your machine per second	Unexpected large upload spike: your data may be being stolen (exfiltrated) to an attacker's server
Network Download (MB/s)	How much data is arriving on your machine per second	Large download while nothing is being installed: malware downloading additional tools (called "staging")
System Uptime	How long since the computer was last restarted	Some implants/rootkits activate after a reboot; very long uptimes with unusual activity may indicate a persistent compromise
Process Count	Number of programs currently running	Sudden large increase: malware that spawns many child processes (e.g. a worm spreading itself)

The Technology Behind Live Telemetry — WebSockets

Normal web pages use HTTP — a one-way conversation where the browser asks a question and the server answers, then the connection closes. To get new data, the browser must ask again (refresh).

HIDS Sentinel uses **WebSocket** instead — a two-way permanent connection that stays open. Think of it as a phone call rather than sending letters. Once the browser "dials in", the server can push data at any moment without waiting to be asked. This is why the graphs update in real-time and alert notifications appear immediately — the server is constantly talking to the browser through this always-open channel.

> 📊 **How Static Hardware Info is Also Collected at Startup**
>
> When the server first starts, `monitor.py` also reads static machine information from the Windows Registry (no admin rights needed): CPU model name (`HKLM\HARDWARE\DESCRIPTION\System\CentralProcessor\0`), total RAM, system manufacturer (e.g. "ASUSTeK COMPUTER INC."), and model name (e.g. "ROG Strix"). This data only changes if hardware is replaced, so it is cached once at startup and served instantly to the dashboard — no repeated reads needed.

8. Machine Learning — How AI is Used to Detect Threats

🧠 **What is Machine Learning — The Non-Technical Explanation**

Machine learning means teaching a computer to recognise patterns by showing it examples, rather than writing out every possible rule by hand. A spam email filter that learned to recognise junk mail by reading 100,000 examples of spam and non-spam is machine learning. A face recognition system that learned what faces look like from millions of photos is ML. In HIDS Sentinel, we teach the ML model what "normal" computer processes look like — so that when something abnormal runs, the model can flag it, even if that specific malware has never been seen before (a "zero-day" threat).

The Three-Layer Threat Intelligence System

Process analysis in HIDS Sentinel uses three complementary layers. If the first layer flags a process, the result is used. If not, it falls through to the next layer. Each layer has different strengths:

🤖 **Layer 1 — Google Gemini AI (Cloud-based Contextual Intelligence)**

Model: gemini-2.5-flash via Google's Generative Language API
When used: When a GEMINI_API_KEY environment variable is configured
How it works: For each candidate process (one that triggered initial suspicion), the system sends a JSON payload to Gemini containing: process name, full command-line arguments, CPU %, memory %, and the host context (machine name, OS version). Gemini's prompt instructs it to act as a "host intrusion detection assistant" and return a structured JSON response with `suspicious` (true/false), `confidence` (0–100), `category` (e.g. "high-risk tooling"), and a plain-English `reason`. Temperature is set to 0.1 to minimise hallucination — the response is factual and deterministic. **Advantage:** Understands context — knows that PowerShell running `Get-Process` is different from PowerShell running `-EncodedCommand ZQBjAG...` (encoded attack payload).

🌲 **Layer 2 — Isolation Forest (Local ML Model, No Internet Needed)**

Model: `sklearn.ensemble.IsolationForest` with `contamination=0.15`
When used: When no Gemini API key is available (offline mode)
What is Isolation Forest? Isolation Forest is an unsupervised anomaly detection algorithm. It works by randomly partitioning data points using decision trees. Normal (common) data points require many partitions to isolate — they are "buried" in clusters with similar data. Anomalous (rare/unusual) points are isolated in very few steps because they have unusual feature values and don't cluster with anything else. The algorithm assigns an anomaly score — the further below zero, the more anomalous.

The 6 features extracted from each process:
[1] CPU usage % [2] Memory usage % [3] Command-line length (longer commands = more suspicious) [4] High-risk tool flag (1 if name is mimikatz/psexec/rclone, else 0) [5] Shell process flag (1 if cmd.exe/powershell.exe) [6] Suspicious argument count (count of patterns like "-encodedcommand", "-w hidden", "downloadstring")

Training data: Trained on 8 synthetic vectors representing typical benign processes (e.g. [0.5% CPU, 1.2% RAM, cmd_len=25, no flags]). Contamination=0.15 means the model expects roughly 15% of processes it sees to be anomalous.

Output: If prediction == -1 (anomaly), returns suspicious=True with a confidence score calculated from the anomaly score: `confidence = min(98, max(72, 85 - (score × 50)))`

📋 **Layer 3 — Rule-Based Fallback (Zero Dependencies)**

When used: If both Gemini and Isolation Forest are unavailable, or for quick pre-screening

How it works: A curated dictionary of known high-risk tool names (`mimikatz.exe` , `psexec.exe` , `rclone.exe` , `certutil.exe` , `mshta.exe` , etc.) and suspicious command-line argument patterns (`-encodedcommand` , `-noprofile -w hidden` , `downloadstring` , `bypass`) are checked against each process. Matches return immediate suspicious=True with confidence 74–92% based on how specific the match is. This layer requires no machine learning libraries and works instantly — it handles the most obvious threats.

📊 **Statistical Baseline Anomaly Scorer (For Events, Not Processes)**

Module: `detector.py` — ThreatDetector class

How it works: When any security event is submitted (e.g. login failure, file modification, network connection), the system compares 5 numerical features of that event against a baseline built from the last 1,000 stored events: failed_login_count, file_modifications, outbound_connections, data_transfer_mb, new_services_installed.

For each feature: `z_score = |current_value - baseline_mean| / baseline_std`

If z_score ≥ 2.5 (2.5 standard deviations above the average), the feature is flagged as anomalous and a reason is added. This is exactly the same statistical method used in financial fraud detection — a credit card company flags a transaction if your spending suddenly deviates by 2.5+ standard deviations from your normal pattern.

How the Final Alert Score is Calculated

The `ThreatDetector.analyze()` method combines both scores mathematically:

Total Score = (Statistical Anomaly Score × 0.45) + (Rule-Based Score × 0.75)

The rule-based component has a higher weight (0.75) because explicit rule matches (e.g. a known hacker tool running) are more reliable than statistical anomalies. The anomaly component (0.45) still contributes significantly — it catches behaviours that rules haven't anticipated.

Severity thresholds: Score ≥ 75 → Critical | ≥ 55 → High | ≥ 35 → Medium | Below 35 → Low

4. How HIDS Sentinel Differs from Existing Security Products

The cybersecurity software market is divided into two categories: consumer products (affordable but limited) and enterprise tools (powerful but extremely expensive). HIDS Sentinel is designed to fill the gap between them.

Consumer Products (Antivirus Software)

Products like Windows Defender, Norton, Kaspersky, and Malwarebytes are designed for ordinary home users. They are easy to install and affordable (or free), but they have significant limitations:

- They rely almost entirely on **signature databases** — they can only detect malware they have already seen. A brand-new malware variant (called a "zero-day") can bypass them completely.
- They do not inspect **firmware or hardware** — they cannot tell you if your BIOS is outdated, your TPM is missing, or Secure Boot is disabled.
- They produce **no security score or posture report** — you have no way to know how exposed your machine is overall.
- They do not map threats to **MITRE ATT&CK** techniques.
- They do not provide a **live dashboard** — there is no way to watch CPU/network graphs and correlate spikes with processes.

Enterprise SIEM Tools

Products like Splunk, IBM QRadar, Microsoft Sentinel, and Elastic SIEM are used by large corporations. They are extremely powerful but completely inaccessible for individuals and small organisations:

- **Cost:** Splunk Enterprise costs ₹10–50 Lakh per year. Microsoft Sentinel charges per gigabyte of data ingested. These are not options for a student, clinic, or small business.
- **Complexity:** Enterprise SIEMs require weeks of configuration, dedicated servers, and a trained security team to operate. A small business owner cannot set one up.
- **No local hardware inspection:** These tools analyse log files and network traffic — they generally cannot inspect your BIOS, read your TPM chip, or tell you about your startup programs.

Full Feature Comparison Table

Capability	Windows Defender / Antivirus	Enterprise SIEM (Splunk etc.)	HIDS Sentinel
Cost	Free – ₹8,000/yr	₹10–50 Lakh/yr	Free / Open Source
Zero-day (unknown malware) detection	❌ Signature only	✅ (expensive ML add-ons)	✅ Isolation Forest ML
BIOS / Firmware inspection	❌	❌	✅ Registry + UAC scan
TPM chip status check	❌	❌	✅ PnP + Elevated
Secure Boot verification	❌	❌	✅ Registry + UAC
Live browser dashboard	❌	✅ (complex setup)	✅ No setup needed
Security Posture Score	❌	❌	✅ 0–100 with grade
MITRE ATT&CK tagging	❌	✅ (advanced config)	✅ Automatic

Capability	Windows Defender / Antivirus	Enterprise SIEM (Splunk etc.)	HIDS Sentinel
YARA malware scanning	❌	Add-on only	✅ Built-in
AI process analysis	❌	Expensive add-on	✅ Gemini + Isolation Forest
File Integrity Monitoring	❌ (basic only)	✅ (with agent)	✅ Built-in FIM
SOAR auto-response	❌	✅ (expensive)	✅ Built-in
Setup time	Minutes	Weeks	One command
Works fully offline	Partially	❌ Cloud dependent	✅ Full offline mode
Required IT expertise	None	Full security team	Basic computer skills

🎯 The Market Position — Layman Summary

HIDS Sentinel occupies a unique position: it has the simplicity and low cost of a consumer antivirus but the depth and intelligence of an enterprise security tool. It is the only freely available solution that combines real-time ML process analysis, firmware-level inspection, YARA malware scanning, SOAR automation, MITRE ATT&CK tagging, and a live browser dashboard — all accessible to anyone who can run a Python script.

7. Where HIDS Sentinel Can Be Used in Real Life

HIDS Sentinel is a practical tool that addresses real security needs across many different types of organisations and individuals. Here are detailed use-case scenarios:

🏥 Scenario 1 — A Healthcare Clinic

> **The Problem Without HIDS Sentinel**
>
> In 2023, AIIMS Delhi was hit by ransomware. The attack encrypted patient records, disrupted emergency services, and took weeks to recover from. The early warning signs — unusual disk write activity, a new suspicious process, registry modifications — were all there but no one was watching for them.

With HIDS Sentinel running on clinic computers: The File Integrity Monitor would have fired an alert the moment the ransomware began encrypting the first files (disk write spike + file hash changes). The Process Scanner would have flagged the ransomware process as anomalous because its CPU usage, disk write rate, and command patterns do not match any normal office software. The SOAR module could automatically terminate the process before the encryption spread. The administrator would receive an immediate alert on their phone. Total damage: potentially zero.

🎓 Scenario 2 — A University Computer Lab

University labs contain student research data, administrative databases, and often serve as launching pads for attacks on the wider university network. Common threats include students installing cryptocurrency miners (which use 100% CPU, costing the university electricity and slowing machines), or attackers compromising lab machines to access the university network.

HIDS Sentinel's Telemetry dashboard would immediately show the CPU spike from a crypto-miner. The Persistence Scanner would catch the miner registering itself in the Registry Run key to survive reboots. The Network Scanner would show the miner's mining pool connection as an unusual outbound connection on port 3333 (the standard mining pool port).

🏢 Scenario 3 — Small Business / Startup

A small startup has 10 employees sharing a Windows server and several work laptops. They cannot afford a dedicated IT security team. An attacker sends a phishing email — an employee clicks a malicious attachment that installs a remote access tool. Without HIDS Sentinel: the attacker has silent, persistent access. With HIDS Sentinel: The new process spawned by the attachment is flagged by the AI classifier. The new Registry Run key added for persistence is caught by the Persistence Scanner. The outbound connection to the attacker's command-and-control server is shown in the Network Scanner. The MITRE ATT&CK tag "T1059 Command Execution" is applied to the alert.

🏛️ Scenario 4 — Government / Public Sector Offices

Government computers store sensitive citizen data, policy documents, and classified information. An insider threat (a disgruntled employee copying files to a USB drive or uploading to the cloud) would be detected by: the Network Scanner (large upload spike), File Integrity Monitor (mass file reads), and USB device logging (the Device Scanner recording the USB insertion event with timestamp, device name, and serial number). The SOAR module can immediately block further USB storage access on that machine.

👨‍💻 Scenario 5 — Individual Developers, Journalists, and Activists

High-value individuals who store sensitive information on personal computers are frequent targets of commercial spyware (like Pegasus or FinFisher). HIDS Sentinel's Firmware Scanner would detect if BIOS settings are tampered with. The Process Scanner would flag spyware processes that access the microphone or camera in the background. The FIM module would alert if any critical system file is modified — a common sign of rootkit installation.

🔬 Scenario 6 — Cybersecurity Education and Training

HIDS Sentinel is an excellent learning tool for cybersecurity students. Instructors can:

- Use the YARA scanner to scan sample malware files downloaded in a controlled lab environment and observe exactly which patterns matched and why
- Use the Event Simulator to generate fake security events (brute-force login, privilege escalation) and watch how the detector calculates scores and creates alerts in real-time
- Study the MITRE ATT&CK mappings in alerts to understand how academic techniques map to real-world events
- Examine the ML classifier's confidence scores and understand why it flagged or cleared specific processes

4. Future Implementation — Where the Project Goes Next

HIDS Sentinel version 1 is a fully functional security monitoring system for a single Windows machine. The following roadmap outlines planned improvements in priority order, from most important to most aspirational:

🚀 Phase 2 — Near-Term (3–6 Months)

🌐 Multi-Machine Agent Network

Currently, HIDS Sentinel monitors the machine it runs on. The agent framework (agent_manager.py) is already built but not yet deployed. Phase 2 will add lightweight Python agents that can be installed on any machine in the network. Each agent sends its telemetry and scan results to the central HIDS Sentinel server, allowing one administrator to monitor 50 machines from a single dashboard. Each machine appears as a card in the "Agent Network" view, showing its IP, OS, online status, and posture score.

📲 Mobile Push Notifications

When a Critical alert fires at 3 AM, no one is watching the dashboard. Phase 2 will integrate Firebase Cloud Messaging (FCM) to send push notifications to a companion Android/iOS app. The notification will show the alert severity, machine name, and a one-line summary — allowing the security administrator to take immediate action even when away from the office.

🧠 Trained ML Model (Real Data, Not Synthetic)

The current Isolation Forest is trained on 8 synthetic data points — adequate for a prototype but not optimal for production. Phase 2 will add a training pipeline that collects real process snapshots from the monitored machine over the first 7 days, labels them as benign (no threats detected), and retrains the model. The result is a personalised ML model that knows exactly what "normal" looks like on that specific machine — drastically reducing false positives.

🔁 Automated Incident Response Playbooks

The current SOAR module can terminate a process or block an IP. Phase 2 will introduce full "playbooks" — pre-defined automated response sequences for specific attack types. Example Ransomware Playbook: (1) Terminate flagged process, (2) Snapshot running processes to a log, (3) Isolate the machine from the network by disabling the network adapter, (4) Alert administrator with full context, (5) Begin damage assessment scan. All within seconds — no human needed.

🔭 Phase 3 — Medium-Term (6–18 Months)

🐧 Linux & macOS Support

The current system is Windows-only because it uses Windows Registry, PowerShell, and Win32 APIs. Linux support will replace these with: /proc filesystem reads (process data), journald/syslog for event logs, and systemd for startup services. macOS support will use launchd for persistence and IOKit for hardware. The React dashboard and MongoDB backend are already cross-platform.

☁️ Cloud Event Archive

If an attacker compromises a machine completely, the first thing they do is delete the local logs. Phase 3 will add encrypted cloud synchronisation of all events to a user-controlled cloud storage (AWS S3 or similar) in near real-time. Even if the local machine is wiped, a forensic record exists in the cloud — admissible as evidence and useful for incident reconstruction.

🕵️ Natural Language Threat Hunting

Security analysts currently query MongoDB directly using technical syntax. Phase 3 will add a natural language interface powered by a language model: type "Show me all PowerShell processes that ran after midnight this week" and the system converts it to a MongoDB query automatically. This makes the tool accessible to non-technical security managers who

📊 Compliance Report Generation

Many businesses must comply with security standards: ISO 27001, NIST Cybersecurity Framework, PCI-DSS (for payment card handling), or India's DPDP Act. Phase 3 will map every HIDS Sentinel finding and control to the relevant compliance framework and automatically generate a gap analysis report — showing which requirements are met and which need

need to investigate incidents but cannot write database queries.

attention. This would make HIDS Sentinel directly useful in preparing for compliance audits.

🌟 Phase 4 — Long-Term Vision (18+ Months)

🤝 Threat Intelligence Feed Integration

Integrate with open threat intelligence sources (VirusTotal, AlienVault OTX, MISP) to automatically check file hashes, IP addresses, and domain names against global threat databases. A process connecting to an IP address already known to be a command-and-control server would be flagged immediately, before any local analysis is even needed.

🏢 SaaS / Multi-Tenant Deployment

Package HIDS Sentinel as a Software-as-a-Service product where managed security service providers (MSSPs) can offer it to multiple client organisations simultaneously. Each organisation gets isolated data storage and a customised dashboard with their branding. This is the commercial productisation path for the project.

Complete Project Summary — At a Glance

🛠 Built With

- Python 3.13 — backend language
- FastAPI — web server framework
- React.js (Vite) — browser dashboard
- MongoDB Atlas — event/alert database
- scikit-learn — Isolation Forest ML
- Google Gemini 2.5 Flash — AI analysis
- YARA — malware rule engine
- bcrypt + JWT — authentication
- WebSocket — live data streaming
- ReportLab + python-docx — PDF/Word reports

✅ Threats Detected

- Known malware (YARA signatures)
- Unknown malware (Isolation Forest ML)
- Firmware/BIOS tampering
- Brute-force login attacks
- Ransomware (FIM + disk spike)
- Suspicious startup entries
- Data exfiltration (network upload spike)
- Privilege escalation
- Malicious scheduled tasks
- Vulnerable security settings

📈 By The Numbers

- 19 Python backend modules
- 8 independent security scanners
- 3 AI/ML detection layers
- MITRE ATT&CK — 14 tactic categories
- Security score: 0–100 with A–F grade
- Live update every 3 seconds
- Posture findings: clickable + detailed popup
- YARA rules: 4 malware families
- 6 ML features per process analysed
- 40+ REST API endpoints

The Three Key Innovations

🔬 Innovation 1 — Firmware-Level Security Visibility (No Equivalent in Consumer Tools)

HIDS Sentinel is one of very few free tools that inspects the BIOS firmware, TPM chip, and Secure Boot state directly from the Windows Registry — no external tools required. With UAC elevation, it accesses full hardware-level security data that even paid antivirus tools cannot see. This matters because modern "firmware implants" (like the Equation Group's tools exposed by Snowden documents) hide below the operating system where no antivirus can reach — only a firmware inspector can detect them.

🧠 Innovation 2 — Dual-Intelligence Process Analysis (ML + Generative AI)

The combination of a local unsupervised ML model (Isolation Forest) with a cloud LLM (Gemini 2.5 Flash) for process threat analysis is a novel approach. The local model catches statistical anomalies instantly with zero internet dependency. The cloud AI adds contextual reasoning that no rule set can replicate — understanding, for example, that the same PowerShell process is benign in one context and an attack in another. The fallback chain (Gemini → Isolation Forest → Rule-based) ensures no single point of failure.

👆 Innovation 3 — Clickable Security Findings with Exact Item Drill-Down

Traditional security tools display alert counts (e.g. "1 suspicious startup entry"). HIDS Sentinel makes every finding card clickable — clicking reveals the exact item: name, registry key location, full command-line path, and why it was flagged. This "drill-down" approach means even a non-technical user can immediately see what to remove and where to find it, rather than needing to know how to navigate the Windows Registry themselves.

Conclusion

HIDS Sentinel demonstrates that enterprise-grade host security monitoring does not require an enterprise budget or an expert team. By combining modern Python libraries, a React.js live dashboard, machine learning anomaly detection, generative AI process intelligence, YARA malware scanning, and deep Windows system introspection — all in a single open-source project — the team has built a tool that is genuinely useful in real-world security scenarios.

The project serves as both a practical security tool and an academic demonstration of how multiple data science disciplines (real-time data streaming, unsupervised machine learning, statistical anomaly detection, API development, and natural language AI) can be integrated into a single, coherent application that solves a meaningful real-world problem.

> 🎯 **One-Sentence Summary for Any Audience**
>
> HIDS Sentinel is a free, AI-powered security dashboard that gives any Windows user the same level of threat detection and visibility that large corporations spend millions of rupees to achieve — all accessible from a web browser in under 5 minutes.